

Guía de Desarrollo: Marca Personal FP (Parte 4)

Capa API y Seguridad. Misma estructura en cada apartado:

- ¿Qué es? → Teoría mínima.
- Ejemplo genérico → Código de muestra (tienda de **Videojuegos**). **NO lo copies al proyecto.**
- Reto del proyecto → Lo que TÚ programas. Te doy comandos y guía, **no el código.**

Requisito previo: Modelos, servicios y controladores web (Bloques 3 y 4) terminados.

BLOQUE 6: API REST (CRUD)

6.1. Rutas y Controladores API

¿Qué es y para qué sirve?

- Una **API REST** devuelve **JSON** (no vistas HTML). La consume un frontend, una app móvil u otro servicio.
- Las rutas API viven en **routes/api.php** y se sirven bajo el prefijo **/api**.
- Es **stateless** (sin sesión): la autenticación se hace con **token**, no con cookies. En Laravel se usa **Sanctum**.

Atención (Laravel 11/12): El archivo **routes/api.php** ya no viene por defecto. Debes generarlo e instalar Sanctum con un comando.

Ejemplo genérico (tienda de Videojuegos)

1. Instalar la capa API (crea **routes/api.php** + Sanctum):

```
Shell  
php artisan install:api
```

2. Crear un controlador **API** (sin **create** ni **edit**, porque no hay formularios):

Shell

```
php artisan make:controller Api/VideojuegoController --api
```

PHP

```
namespace App\Http\Controllers\Api;

class VideojuegoController extends Controller
{
    public function index()
    {
        return Videojuego::paginate(15); // se serializa a JSON
solo
    }

    public function store(Request $request)
    {
        $datos = $request->validate([
            'titulo' => 'required|string',
            'precio' => 'required|numeric',
        ]);

        $videojuego = Videojuego::create($datos);
        return response()->json($videojuego, 201); // 201 =
creado
    }

    public function destroy(Videojuego $videojuego)
    {
        $videojuego->delete();
        return response()->json(null, 204); // 204 = sin
contenido
    }
}
```

3. Registrar la ruta en `routes/api.php` y protegerla con token:

PHP

```
use App\Http\Controllers\Api\VideojuegoController;
```

```
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('videojuegos',
VideojuegoController::class);
});
```

Pistas: `apiResource` crea 5 rutas (index, store, show, update, destroy), sin create/edit. Devuelve **códigos HTTP correctos**: 200 ok, 201 creado, 204 borrado, 422 validación.

Reto del proyecto

1. Instala la capa API:

```
Shell
php artisan install:api
```

2. Crea un controlador API por entidad dentro de `App\Http\Controllers\Api`:

```
Shell
php artisan make:controller Api/NombreController --api
```

Controlador API	Recurso
<code>Api/ProyectoController</code>	proyectos
<code>Api/EstudianteController</code>	estudiantes
<code>Api/CicloFormativoController</code>	ciclos

3.

Registra las rutas en `routes/api.php` con `apiResource`, dentro de un grupo `auth:sanctum`.

4. Implementa los 5 métodos delegando en tus **Services** (los mismos del Bloque 3):

Método	Devuelve
<code>index</code>	Listado paginado (JSON)

`store` Registro creado + código 201

`show` Un registro (JSON)

`update` Registro actualizado

`destroy` Vacío + código 204 y

Pistas importantes:

- **Reutiliza los Services:** no dupliques lógica entre el controlador web y el API.
- **Valida siempre** (`$request->validate`); en API, los errores salen automáticamente como JSON con código 422.
- Prueba los endpoints con **Postman** o `curl` antes de seguir.

Objetivo: CRUD completo accesible en `/api/proyectos`, `/api/estudiantes`, etc., devolviendo JSON.

6.2. API Resources (formato de salida JSON)

¿Qué es y para qué sirve?

- Un **API Resource** es una clase que decide **cómo se transforma un modelo a JSON**.
- Evita exponer campos sensibles y da una **estructura uniforme** a todas las respuestas.
- Sin él, devuelves el modelo "crudo"; con él, controlas exactamente qué se publica.

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:resource VideojuegoResource
```

PHP

```
public function toArray(Request $request): array
{
    return [
        'id' => $this->id,
        'titulo' => $this->titulo,
```

```

        'precio' => $this->precio,
        // 'stock' NO se expone, por ejemplo
        'categorias' => $this->categorias->pluck('nombre'),
    ];
}

```

Uso en el controlador:

```

PHP
return VideojuegoResource::collection(Videojuego::paginate(15));
// listado
return new VideojuegoResource($videojuego);
// uno solo

```

Reto del proyecto

1. Crea un Resource por entidad:

```

Shell
php artisan make:resource NombreResource

```

Resource	Qué exponer (decide tú)
ProyectoResource	id, nombre, descripcion, dificultad, sus estudiantes
EstudianteResource	id, nombre, apellidos, sus ciclos (sin DNI/datos sensibles)
CicloFormativoResource	id, nombre, grado, su familia profesional

2. Usa los Resources en los controladores API en vez de devolver el modelo directo.

Pistas importantes:

- Oculta datos sensibles (DNI, email, etc.) si no deben publicarse.
- Para incluir relaciones, recórrelas dentro del `toArray` (ej. `$this->estudiantes`).

Objetivo: Todas las respuestas API tienen un formato JSON limpio y controlado.

BLOQUE 7: Autorización (Políticas y Roles)

7.1. Roles y Políticas

¿Qué es y para qué sirve?

- **Autenticar** = saber **quién eres** (login, Bloque 1). **Autorizar** = saber **qué puedes hacer**.
- Una **Policy** es una clase con las **reglas de permiso de un modelo**. Cada método responde **true/false**.
- Laravel **descubre la policy automáticamente** por convención: **ProyectoPolicy** → modelo **Proyecto**.
- En *Marca Personal FP* hay **roles** (ej. **estudiante**, **docente**, **admin**) que determinan los permisos.

Ejemplo genérico (tienda de Videojuegos)

1. Suponemos un campo **role** en **users** (**admin** / **cliente**):

PHP

```
// migración nueva sobre la tabla users
$table->string('role')->default('cliente');
```

2. Crear la policy ligada al modelo:

Shell

```
php artisan make:policy VideojuegoPolicy --model=Videojuego
```

PHP

```
class VideojuegoPolicy
{
    public function update(User $user, Videojuego $videojuego):
    bool
    {
        // Solo el dueño o un admin pueden editar
        return $user->id === $videojuego->user_id || $user->role
        === 'admin';
    }
}
```

```

    public function delete(User $user, Videojuego $videojuego):
bool
    {
        return $user->role === 'admin';
    }
}

```

Pista: Métodos típicos de una policy: `viewAny`, `view`, `create`, `update`, `delete`.

Reto del proyecto

1. **Define cómo identificas el rol** (elige una opción):
 - Añadir un campo `role` a la tabla `users` con una migración nueva, o
 - Deducir el rol según si el user tiene relación `estudiante` o `docente`.
2. **Crea una policy por entidad protegida:**

Shell

```
php artisan make:policy NombrePolicy --model=NombreModelo
```

Policy	Reglas a implementar (ejemplos)
<code>ProyectoPolicy</code>	Editar/borrar: solo el estudiante dueño o un docente/admin
<code>EstudiantePolicy</code>	Editar perfil: solo el propio estudiante o admin
<code>CicloFormativoPolicy</code>	Crear/editar/borrar: solo docente/admin

3. **Programa los métodos** (`viewAny`, `view`, `create`, `update`, `delete`) devolviendo `true/false` según el rol y la propiedad del recurso.

Objetivo: Tener escritas las reglas de "quién puede qué" para cada modelo.

7.2. Aplicar los permisos

AVISO CRÍTICO (Laravel 11 y 12) — evita el fallo tonto más típico: En Laravel 11/12 la clase base `app/Http/Controllers/Controller.php` viene **vacía**. Por eso `$this->authorize(...)` y `$this->validate(...)` **NO existen** y darán el error *"Call to undefined method"*. **Dos soluciones (elige una):**

1. **Añadir los traits a la clase base** (recupera la sintaxis clásica):

PHP

```
// app/Http/Controllers/Controller.php
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
abstract class Controller {
    use AuthorizesRequests, ValidatesRequests;
}
```

2. **No tocar nada y usar la fachada Gate** (funciona siempre, sin traits):

PHP

```
use Illuminate\Support\Facades\Gate;
Gate::authorize('update', $proyecto);
```

Nota: `$request->validate(...)` (sobre el objeto Request) **sí funciona siempre**, no necesita trait. El problema es solo con `$this->validate()` y `$this->authorize()`.

¿Qué es y para qué sirve?

- Una vez escrita la policy, hay que **invocarla** en tres sitios:
 - **Controlador** → `$this->authorize('update', $proyecto)` (lanza 403 si no puede). **Requiere el trait** (ver aviso de arriba) o usar `Gate::authorize(...)`.
 - **Vista Blade** → `@can('update', $proyecto) ... @endcan` (oculta botones). Funciona sin traits.
 - **Rutas/API** → middleware `->can('update', 'proyecto')`. Funciona sin traits.

Ejemplo genérico (tienda de Videojuegos)

En el controlador:

PHP

```
public function update(Request $request, Videojuego $videojuego)
{
```



```
$this->authorize('update', $videojuego); // 403 si no tiene
permiso
// ... lógica de actualización
}
```

En la **vista Blade**:

```
None
@can('update', $videojuego)
    <a href="{{ route('videojuegos.edit', $videojuego)
    }}">Editar</a>
@endcan
```

Reto del proyecto

1. **Protege los controladores** (web y API) con `$this->authorize(...)` en `update`, `destroy` y donde corresponda.
2. **Ocultas los botones** de editar/borrar en las vistas con `@can` / `@cannot`, para que el usuario solo vea lo que puede hacer.
3. **Comprueba los 3 escenarios:**

Usuario	Acción	Resultado esperado
Dueño del recurso	Editar lo suyo	Permitido
Otro estudiante	Editar lo ajeno	403 Prohibido
Docente / admin	Editar cualquiera	Permitido

Pistas importantes:

- `authorize()` corta la ejecución con error **403** si la policy devuelve `false`.
- En la API, un fallo de autorización también devuelve **403** en JSON, sin que tengas que hacer nada extra.
- No confíes solo en ocultar botones (`@can`): la **seguridad real** está en `authorize()` del controlador.

Objetivo: Que cada usuario solo pueda ver y modificar lo que su rol le permite, tanto en web como en API.

Resumen de la capa añadida: Ya tienes una **API REST con CRUD** (con Resources para el JSON) y un **sistema de permisos por roles** con Políticas aplicado en controladores, vistas y rutas. Con esto el proyecto cubre web + API + seguridad.